Business Entity Resolution Challenge

In large-scale commercial platforms, business identity data arrives from multiple independent sources — each contributing partial, noisy fragments of information about the same real-world entities. These fragments share no common identifiers, and the challenge of determining which records refer to the same business is known as Entity Resolution (ER). Your challenge is to build an ML solution that, given business records from 3 independent data sources with noisy and inconsistent fields, determines which records across sources refer to the same real-world business entity.

Source 1 is the deduplicated reference source. Your task is to find all matching records from Source 2 and Source 3 for each Source 1 entity. A Source 1 entity may match zero, one, or many records from Source 2 and Source 3.

Please refer to this video for a better understanding of the problem statement: **Click Here**

File Format

All files in this challenge are tab-separated (`.tsv`), and your submissions must be tab-separated too. Tabs are used because business addresses and the ID list columns both contain commas. Read them with an explicit tab separator, for example:

```
import pandas as pd
df = pd.read_csv("dataset/train/train_source1.tsv", sep="\t")
```

Reading a `.tsv` without `sep="\t"` will silently produce a single column containing the whole line.

Data Description:

Each source file (`*_source1.tsv`, `*_source2.tsv`, `*_source3.tsv`) has the following columns:

1. **entity_id:** Unique identifier for the record. The prefix indicates the source — `S1-`, `S2-`, or `S3-`.
2. **business_name:** Name of the business entity (may contain abbreviations, legal suffixes, typos, transliterations)
3. **business_address:** Address of the business (may contain partial addresses, format variations, missing components, landmark-based references)
4. **country:** Country label for the record. The **training** data covers `US` and `India`. The **test** set additionally contains a third country, `France`, that does **not** appear in the training

data. Treat `country` as an open set of string labels: do **not** hard-code, filter, or one-hot your pipeline to only `{US, India}`, and remember that every test entity — `France` included — must appear in your submission.

There is no separate *source* column — a record's source is given by its `entity_id` prefix (`S1-`/`S2-`/`S3-`) and by which file it appears in.

The ground truth file (`train_ground_truth.tsv`) has two columns:

1. **source1_entity_id:** The `entity_id` of a Source 1 record
2. **matched_entity_ids:** Comma-separated list of matching `entity_id`s from Source 2 and/or Source 3 (empty when the entity has no matches)

Noise Patterns to Expect:

- **Name variations:** Abbreviations (Corp vs. Corporation, Pvt vs. Private, Ltd vs. Limited), legal suffix inconsistencies, DBA/trade names, punctuation differences (& vs. "and"), word-order transpositions, typos
- **Address variations:** Abbreviations (Rd vs. Road, St vs. Street), transliteration variants, missing components (no PIN code, no state), landmark-based references (Near SBI ATM), municipal numbering formats, component reordering

Dataset Details:

- **Training Dataset:** Business records across 3 sources with ground truth matching labels
- **Test Set:** Business records across 3 sources without matching labels

File Descriptions:

Training files

1. **dataset/train/train_source1.tsv:** Source 1 training records (the deduplicated reference source)
2. **dataset/train/train_source2.tsv:** Source 2 training records
3. **dataset/train/train_source3.tsv:** Source 3 training records
4. **dataset/train/train_ground_truth.tsv:** Ground truth matching labels for the training set

Test files

1. **dataset/test/test_source1.tsv:** Source 1 test records. Generate matches for every entity in this file.
2. **dataset/test/test_source2.tsv:** Source 2 test records
3. **dataset/test/test_source3.tsv:** Source 3 test records

No ground truth is provided for the test set. To measure your own performance, hold out a validation split from the training data and score it yourself using the F_0.5 formula given below.

Output Format:

Your solution produces **two** tab-separated files, both placed in the `output/` folder of your final submission package (see *Final Submission Package* below):

1. **`matching_results.tsv`** — your final entity matches. **This is the only file scored on the leaderboard** — it is what you upload to the Portal during the challenge.
2. **`candidate_pairs.tsv`** — the candidate set your blocking / candidate-generation stage produced, before your final matching model narrowed it down.

matching_results.tsv

Your final entity matches:

Column	Description
source1_entity_id	The `entity_id` of a Source 1 record
matched_entity_ids	Comma-separated list of matching `entity_id`s from Source 2 and/or Source 3

Example (columns separated by a single tab, ID lists separated by commas with no quoting):

```
source1_entity_id	matched_entity_ids
S1-00001	S2-00047,S2-00193,S3-00812
S1-00002	S3-00004
S1-00003
```

Important:

- Every Source 1 entity in the test set must have exactly one row
- Leave `matched_entity_ids` empty for entities with no matches (singletons)
- No duplicate entity IDs within a single ID list
- ID lists must only contain Source 2 or Source 3 IDs that exist in the test set

candidate_pairs.tsv

The candidate set from your blocking stage — every Source 2 / Source 3 record you considered a plausible match for each Source 1 entity, *before* your final matching model narrowed it down. This is the **exact set of records you feed into your matching model for inference** — the final candidate list *just before* the ML model scores them, not the raw output of an early blocking

pass you later filter further. If your pipeline has several blocking/filtering stages, `candidate_pairs.tsv` is the *last* one: whatever your model actually runs inference over. Every ID in `matching_results.tsv` should therefore appear here.

It is **not scored on the leaderboard**; we use it to analyse blocking quality (recall ceiling, reduction ratio) and to verify your pipeline.

Column	Description
source1_entity_id	The `entity_id` of a Source 1 record
candidate_entity_ids	Comma-separated list of candidate `entity_id`s from Source 2 and/or Source 3

Example:

source1_entity_id	candidate_entity_ids
S1-00001	S2-00047,S2-00193,S3-00812,S3-00999
S1-00002	S3-00004
S1-00003	

Same rules as `matching_results.tsv`: one row per Source 1 entity, `candidate_entity_ids` empty when blocking found no candidates, S2-/S3- IDs only, no duplicates within a list. Your final matches should be a **subset** of your candidates (a matched ID that never appeared as a candidate signals a pipeline bug — the validator warns about it).

Validate before submitting: a helper script `utils/validate_submission.py` (stdlib only, no dependencies) checks both files against every rule above so you can catch a rejection locally instead of spending a submission on it. Run it from this `student_resource/` directory:

```
python3 utils/validate_submission.py \
  --matching output/matching_results.tsv \
  --candidate output/candidate_pairs.tsv \
  --test-dir dataset/test
```

It prints `PASS` (exit 0) when the files are safe to submit, or a numbered list of issues to fix (exit 1). It only reads your output files and the test source files; it does not compute your score.

Final Submission Package:

In addition to your live leaderboard uploads, **every team submits a single zip archive** with your code and outputs. We use it to reproduce your results, audit your blocking, and check the

fair-play and model-license rules — the top teams' packages are reviewed in detail before the final rankings are confirmed.

Structure:

```
<team_name>_submission.zip
├── output/
│   ├── matching_results.tsv       # final matches (same file you upload to the leaderboard)
│   └── candidate_pairs.tsv        # your blocking candidate set
├── code/
│   └── business_entity_resolution/
│       ├── src/                   # all your source code
│       ├── README.md              # how to reproduce end-to-end (data → blocking → matching
→ output)
│       └── requirements.txt       # pinned dependencies / environment
└── Documentation_template.md      # your methodology write-up (this filled-in template)
```

- **`output/`** — the two TSV files described above: `matching_results.tsv` and `candidate_pairs.tsv`.
- **`code/business_entity_resolution/`** — a self-contained, runnable copy of your pipeline. Put all source under `src/`, and include a `README.md` with exact run instructions plus a `requirements.txt` (or equivalent environment file) pinning versions. Anyone should be able to regenerate both output files from the training/test data using only what is in this folder.
- **Methodology document** — fill in the provided `Documentation_template.md` and drop it straight into the zip (the filled-in `.md` is fine; a `.pdf` export works too). No need to rename it.

Constraints:

1. Format your output exactly as described above. Submissions that fail validation will not be evaluated. You should see a `SCORED` status with your F_0.5 score if the output is correctly formatted.
2. `matched_entity_ids` must only reference entities from Source 2 or Source 3. Self-matches to Source 1, and IDs that do not exist in the test set, will be rejected.
3. Every Source 1 entity must appear in your submission. Missing entities will cause rejection.
4. Duplicate entity IDs in any ID list will cause rejection, as will duplicate `source1_entity_id` rows.
5. Final model should be a MIT/Apache 2.0 License model and up to 8 Billion parameters.

Evaluation Criteria:

Submissions are evaluated using **F_β Score (β = 0.5)** — a precision-heavy metric that penalizes false merges (matching two different businesses) more than missed matches.

Formula:

F_0.5 = (1.25 × Precision × Recall) / (0.25 × Precision + Recall)

Computed as a **macro-average**: F_0.5 is calculated per Source 1 entity, then averaged across **all** Source 1 entities in the evaluation set.

Singletons are included in that average. A Source 1 entity with no true matches scores 1.0 when you correctly predict an empty list, and 0.0 when you predict any match for it. Correctly identifying singletons therefore earns credit, and false merges on them are penalised.

Why precision-heavy? In real-world entity resolution, merging two distinct businesses (false positive) is more damaging than missing a link (false negative). F_0.5 weights precision 2× over recall.

Example:

- Your model predicts S1-00001 matches [S2-00047, S2-00193, S3-00812]
- Ground truth says S1-00001 matches [S2-00047, S3-00812]
- Precision = 2/3, Recall = 2/2 = 1.0
- F_0.5 = (1.25 × 0.667 × 1.0) / (0.25 × 0.667 + 1.0) = **0.714**

Leaderboard Information:

- **Public Leaderboard:** During the challenge, rankings will be based on a subset of the test set to provide real-time feedback on your model's performance.
- **Private Leaderboard:** After the challenge ends, the private leaderboard will be revealed, which uses the remaining portion of the test set for evaluation.
- **Final Rankings:** The final decision will be based on the private leaderboard.

You submit predictions for the full test set in both cases; the split is applied during scoring.

Submission Requirements:

1. **Leaderboard (during the challenge):** upload `matching_results.tsv` in the Portal — tab-separated, with the exact column names described above. This is what drives the public and private leaderboards.
2. **Final submission package:** submit the single zip described in *Final Submission Package* above — `output/` with **both** `matching_results.tsv` (final matches) and `candidate_pairs.tsv` (your candidate-generation / blocking set fed to the model), `code/business_entity_resolution/` (runnable pipeline), and your methodology

document. All teams must submit it; the top teams' packages are reviewed before the final rankings are confirmed.

3. Your methodology document must describe:
    - Methodology used
    - Candidate generation/blocking strategy
    - Model architecture and feature engineering
    - Any other relevant information about the approach
    - A template for this documentation is provided in `Documentation_template.md`. There is no page limit — prioritise clarity and technical depth over brevity.

Academic Integrity and Fair Play:

⚠️ STRICTLY PROHIBITED: External Data Lookup

Participants are **STRICTLY NOT ALLOWED** to use external databases, APIs, or services to look up business identities or resolve entities. This includes but is not limited to:

- Using commercial entity resolution APIs or services
- Looking up business registrations from government databases
- Using geocoding APIs to normalize addresses
- Any external data augmentation from internet sources

Enforcement:

- All submitted approaches, methodologies, and code pipelines will be thoroughly reviewed and verified
- Any evidence of external data lookup will result in **immediate disqualification**

Fair Play: This challenge is designed to test your machine learning and data science skills using only the provided training data.

Tips for Success:

- Invest in a strong blocking/candidate generation strategy — it determines the upper bound of your recall
- Explore string similarity features (Jaccard, Levenshtein, TF-IDF cosine) for name and address matching
- Pay attention to country-specific address patterns
- Consider the precision-recall trade-off carefully — F_0.5 rewards precision more than recall
- Do not neglect singletons — correctly predicting "no match" is worth a full 1.0 on that entity
- Validate your own output format against the rules above before submitting